

The MLP: Where Facts Live

The part of the model that thinks on its own · video 2:16

Attention moves information between tokens. This lesson is about the other half of every layer: the **MLP**, which works on **each token's vector by itself**. It expands the vector, bends it with a simple nonlinear function, and projects it back down. It holds most of each layer's weights, and much of the model's factual knowledge seems to live there.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. One token at a time	▶ 0:08-0:21	4	☐
2. Expand, bend, project back	▶ 0:21-0:43	4	☐
3. Neurons as detectors	▶ 0:44-1:00	3	☐
4. GELU: quiet unless the pattern shows up	▶ 1:01-1:14	3	☐
5. Why the bend matters	▶ 1:14-1:26	4	☐
6. Recalling facts	▶ 1:26-1:44	3	☐
7. Where the weights are, in code	▶ 1:44-2:14	4	☐

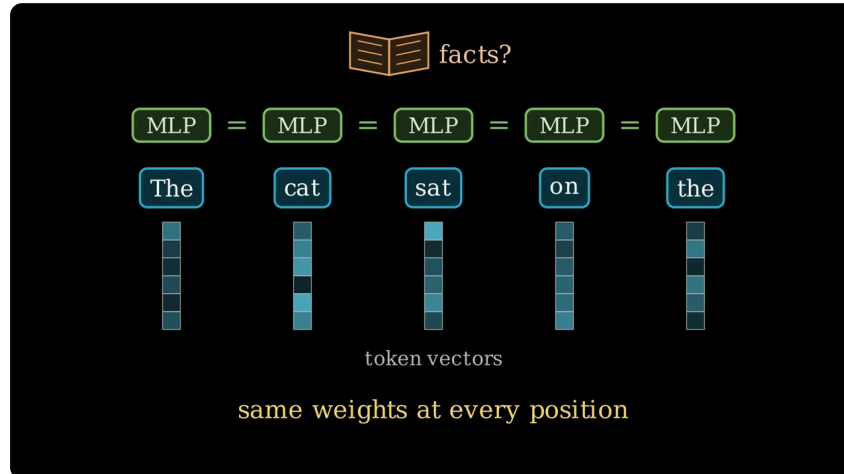
Before you start: this lesson builds on episode 3 (every token is a vector of 768 numbers in GPT-2 small) and episodes 5-7 (attention). The maths is dot products and matrix multiplication: Foundations F02 (The Dot Product), F03 (Matrices), F04 (Straight Lines and Bends) and F12 (Neural Networks in Three Minutes) cover everything used here.

CONCEPT 1 OF 7

One token at a time

▶ Video 0:08-0:21

Attention moves information **between** tokens. The **MLP** does a different kind of work: it takes **each token's vector on its own** and transforms it, without looking at any other token. Every position goes through **the same MLP, with the same weights**.



▶ 0:20 Every token's vector goes through its own copy of the MLP, and every copy has the same weights.

KEY IDEA

Attention: tokens talk to each other. MLP: each token is processed **by itself**, with the **same weights at every position**. It is also where much of the model's **factual knowledge** seems to live (concept 6).

Check 1: One token at a time

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Which sentence describes the MLP?

- A. It moves information between tokens
- B. It processes each token's vector on its own, with the same weights at every position
- C. It uses a different set of weights for each position in the text
- D. It only processes the last token of the text

1.2 TRUE OR FALSE

"Inside the MLP, the vector for *sat* can read information from the vector for *cat*."

True ☐ False ☐

1.3 CALCULATION

A text has 5 tokens. In one layer, (a) how many times is the MLP applied, and (b) how many different sets of MLP weights are used?

1.4 SHORT ANSWER

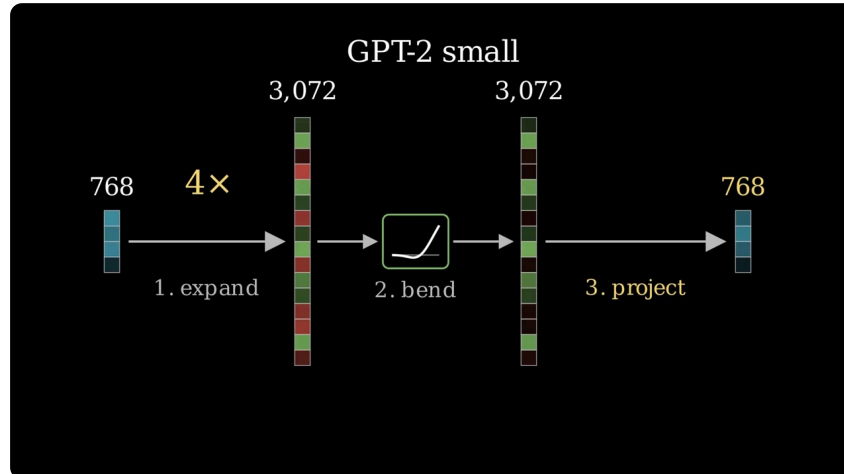
Two tokens reach a layer's MLP with exactly the same vector. What can you say about the two outputs?

CONCEPT 2 OF 7

Expand, bend, project back

▶ Video 0:21-0:43

MLP stands for **multi-layer perceptron**. Here it is just three steps: **expand** the vector to **four times** its size (a matrix multiplication), apply a simple **nonlinear function** to every number (the “bend”), then **project** it back down to its original size (a second matrix multiplication).



▶ 0:43 The three steps, with the sizes in GPT-2 small: 768 numbers in, 3,072 in the middle, 768 out.

In GPT-2 small that is **768 → 3,072 → 768**.

KEY IDEA

MLP = **expand (4x) → bend → project back**. The vector leaves the MLP with the same size it came in with.

Check 2: Expand, bend, project back

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Put the three MLP steps in order: *project back down · expand to 4x · apply the nonlinear function*.

2.2 CALCULATION

GPT-2 small has vectors of 768 numbers. (a) How many numbers does the expanded (hidden) vector have? (b) How many does the MLP's output have?

2.3 CALCULATION

Another model has vectors of 1,024 numbers and uses the same 4x rule. How many numbers are in one token's hidden vector? And in the hidden vectors of a 10-token text, all together?

2.4 TRUE OR FALSE

“The MLP makes the vector permanently bigger: a token enters with 768 numbers and leaves with 3,072.”

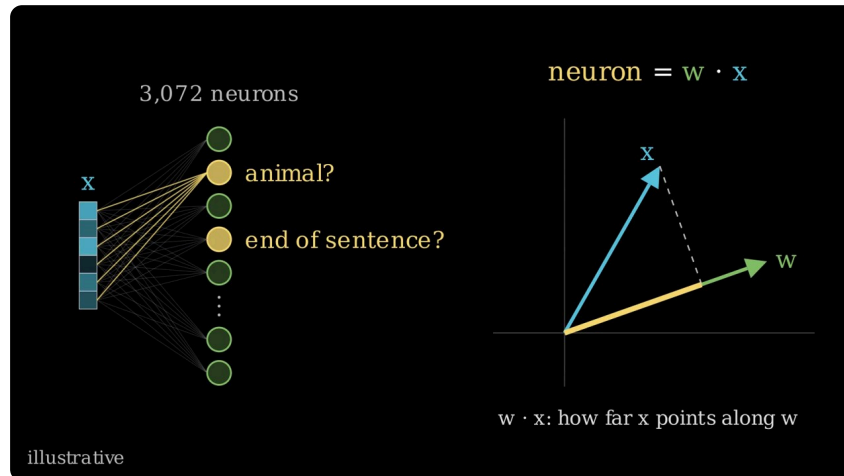
True ☐ False ☐

CONCEPT 3 OF 7

Neurons as detectors

▶ Video 0:44–1:00

Each of the 3,072 hidden numbers is a **neuron**. A neuron has its own weight vector **w** (768 numbers, one column of W_1) and takes a **dot product** with the token's vector: **neuron** = $w \cdot x$ (plus a bias). It measures **how far x points along w**: large when they point the same way, zero at right angles, negative when they point in opposite directions.



▶ 1:00 Each hidden number is a neuron: a dot product $w \cdot x$ that measures how far the token's vector x points along the neuron's direction w .

So each neuron is like a **question** about the token: *is this about animals? Is this the end of a sentence?* (illustrative labels).

KEY IDEA

A neuron is a **dot product** with its own direction w . It gives a big number when the token's vector **points along w**: a detector for one pattern.

Check 3: Neurons as detectors

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

A tiny neuron has $w = [1, 0, 2]$. Compute $w \cdot x$ for $x = [0.5, -1, 1]$ and for $x' = [-1, 2, -1]$. Which of the two tokens matches the neuron's pattern?

3.2 MULTIPLE CHOICE

Four token vectors all have the same length. Which one gives the neuron the largest value?

- A. The one pointing the same way as w
- B. The one pointing the opposite way to w
- C. The one at right angles to w
- D. They all give the same value

3.3 TRUE OR FALSE

"In GPT-2 small, each neuron's weight vector w has 768 numbers, and each MLP has 3,072 neurons."

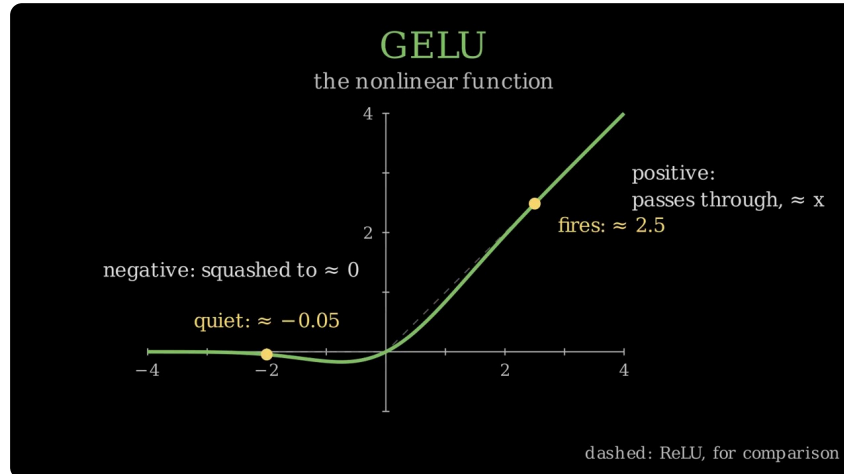
True ☐ False ☐

CONCEPT 4 OF 7

GELU: quiet unless the pattern shows up

▶ Video 1:01-1:14

The nonlinear function is usually **GELU**, applied to every neuron separately. **Large positive** inputs pass through almost unchanged ($\text{GELU}(2.5) \approx 2.5$), and **negative** inputs are squashed close to zero ($\text{GELU}(-2) \approx -0.05$). Around zero it bends smoothly from one behaviour to the other.



▶ 1:13 GELU: positive inputs pass through ($\approx x$), negative inputs are squashed to ≈ 0 . A neuron at -2 stays quiet; one at 2.5 fires.

So a neuron **stays quiet** unless its pattern really shows up: only a clearly positive match is passed on. The dashed line in the video is **ReLU**, $\max(0, x)$, a simpler cousin with a sharp corner at 0.

KEY IDEA

$\text{GELU}(x) \approx x$ for large positive x and ≈ 0 for negative x . A neuron **fires** only when its dot product is clearly positive.

Check 4: GELU: quiet unless the pattern shows up

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which value is closest to $\text{GELU}(3)$?

- A. 0
- B. 1
- C. 3
- D. 9

4.2 CALCULATION

Four neurons have dot products 2.5, -2 , 4 and -3 . Roughly what are the four values after GELU? Which neurons fire?

.....

.....

4.3 TRUE OR FALSE

"GELU turns every negative input into exactly 0."

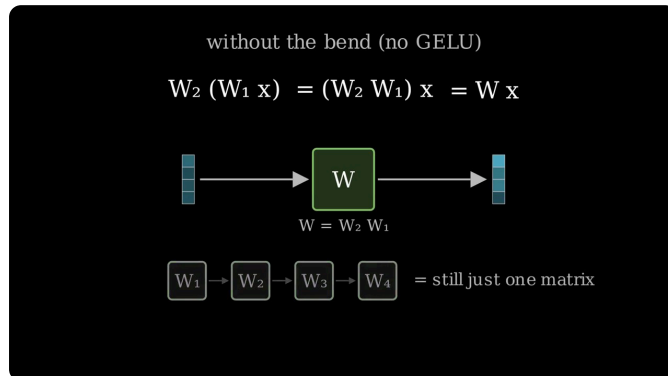
True ☐ False ☐

CONCEPT 5 OF 7

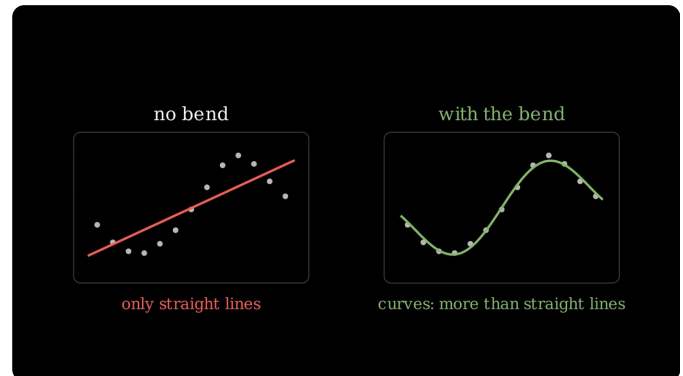
Why the bend matters

▶ Video 1:14-1:26

Without the bend, the MLP would be two matrix multiplications in a row, and those **collapse into one**: $W_2(W_1x) = (W_2W_1)x = Wx$. Stacking more layers would add nothing: a chain of matrices is still one matrix, which can only describe **straight lines**.



▶ 1:21 Without the bend, two matrices collapse into one, and so does any stack of them.



▶ 1:25 No bend: only straight lines. With the bend: curves.

KEY IDEA

A matrix times a matrix is just another matrix. The **nonlinear bend** between them breaks the collapse and lets the network learn **curves**: more than straight lines.

Check 5: Why the bend matters

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Let $W_1 = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ and $W_2 = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$ (written row by row). (a) Compute $W = W_2W_1$. (b) For $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, check that $W_2(W_1x)$ and Wx give the same result.

5.2 SHORT ANSWER

In GPT-2 small, W_1 is $768 \times 3,072$ and W_2 is $3,072 \times 768$. If you removed GELU, the MLP would collapse into a single matrix. What shape, and how many numbers?

5.3 TRUE OR FALSE

“Ten matrix multiplications in a row, with no nonlinearity between them, can learn curves that a single matrix cannot.”

True ☐ False ☐

5.4 MULTIPLE CHOICE

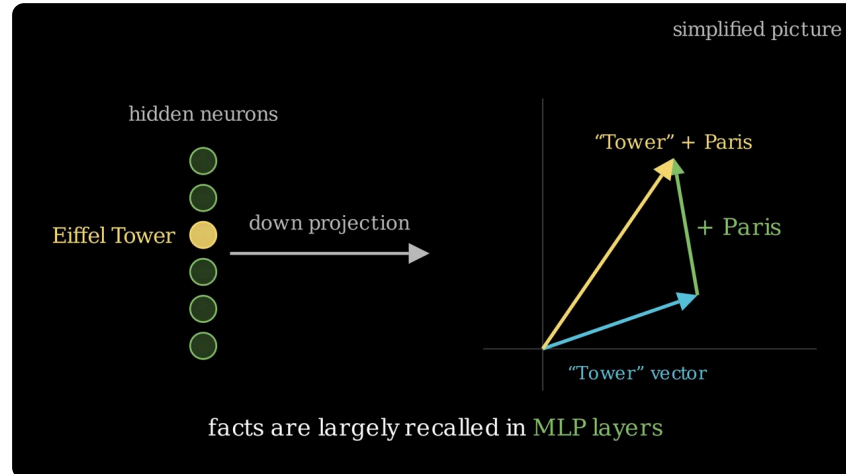
What is the job of the nonlinear function in the MLP?

- A. To make the vector four times bigger
- B. To stop the two matrix multiplications collapsing into one, so the network can learn curves
- C. To turn the vector into probabilities
- D. To move information between tokens

Recalling facts

▶ Video 1:26–1:44

The **down projection** (the second matrix, W_2) turns each **active** neuron into a **push** in some direction. Every neuron has its own output direction (one row of W_2), which is added in proportion to how strongly the neuron fired. Quiet neurons add almost nothing.



▶ 1:43 Simplified picture: the “Eiffel Tower” neuron fires, and the down projection adds a push that means Paris.

A simplified picture: if a neuron that detects *Eiffel Tower* fires, it might add a direction that means *Paris*, so the vector now carries “Tower” + Paris. Researchers have found that facts like this are **largely recalled in the MLP layers**.

KEY IDEA

Detect, then push: a neuron fires on a pattern, and the down projection **adds its direction** (for example, a fact) to the token's vector.

Check 6: Recalling facts

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

After GELU, three neurons have values $h = [2, 0, 0.5]$. Their output directions (rows of W_2) are $[1, 0]$, $[0, 1]$ and $[-2, 2]$. What does the MLP add to the vector?

6.2 MULTIPLE CHOICE

In the simplified picture, what happens when the *Eiffel Tower* neuron fires?

- A. The token is replaced by the word Paris
- B. A direction meaning Paris is added to the token's vector
- C. Attention searches the other tokens for Paris
- D. The model immediately outputs the word Paris

6.3 TRUE OR FALSE

“Researchers have found that facts like *the Eiffel Tower is in Paris* are largely recalled in the attention layers.”

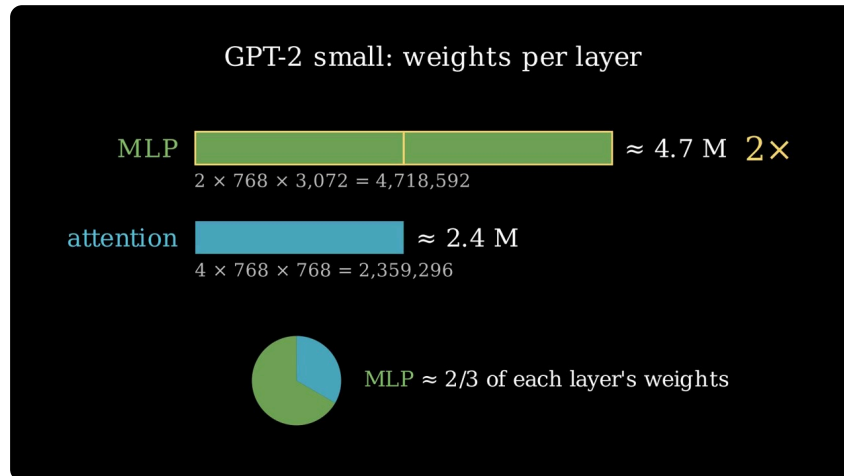
True ☐ False ☐

CONCEPT 7 OF 7

Where the weights are, in code

▶ Video 1:44-2:14

The MLP also holds **most of each layer's weights**. In GPT-2 small its two matrices have $2 \times 768 \times 3,072 = \mathbf{4,718,592}$ weights (≈ 4.7 M), while attention's four 768×768 matrices (query, key, value and output) have $4 \times 768 \times 768 = \mathbf{2,359,296}$ (≈ 2.4 M). That is **twice** as many: the MLP is about **two-thirds** of each layer.



▶ 1:55 Weights per layer in GPT-2 small: the MLP has twice as many as attention.

In code, it is two matrix multiplications with GELU in between: expand and bend, then project back (gelu is written out in exercise 7.4).

```
def mlp(x, W1, b1, W2, b2):  
    h = gelu(x @ W1 + b1)    # expand 768 -> 3072, then bend  
    return h @ W2 + b2       # project back 3072 -> 768
```

So each layer has two halves: **attention, where tokens talk**, and **the MLP, where each token thinks**. Episode 9 wires them together into a transformer block.

KEY IDEA

MLP ≈ 4.7 M weights per layer, attention ≈ 2.4 M: **2x**. In code: `gelu(x @ W1 + b1) @ W2 + b2`.

Check 7: Where the weights are, in code

Answer all 4 questions before looking at the answers. Pass mark: all correct.

7.1 CALCULATION

What fraction of a GPT-2 small layer's weights (MLP + attention) is in the MLP?

7.2 CALCULATION

A model has vectors of 1,024 numbers, a 4x hidden layer, and attention with four 1,024 x 1,024 matrices. How many weights do the MLP and attention each have per layer? Is the MLP still twice as big?

7.3 MULTIPLE CHOICE

In `h = gelu(x @ W1 + b1)`, `x` is one token's vector (768 numbers). What does `h` hold?

- A. 768 numbers: the output of the MLP
- B. 3,072 numbers: one value per neuron, after the bend
- C. 50,257 numbers: one per word in the vocabulary
- D. 12 numbers: one per layer

7.4 HANDS-ON CODE

Try it yourself. The first two functions are the code from the video; the rest builds a random MLP of GPT-2-small size and runs it on 5 tokens at once. (a) What does the first `print` show? (b) What does the second show, and what is the shape of `h` inside `mlp`? (c) The third prints `True`. What does that tell you about how the MLP treats tokens?

```
import numpy as np

def gelu(x):
    c = np.sqrt(2 / np.pi)
    return 0.5 * x * (1 + np.tanh(c * (x + 0.044715 * x**3)))

def mlp(x, W1, b1, W2, b2):
    h = gelu(x @ W1 + b1)      # expand 768 -> 3072, then bend
    return h @ W2 + b2        # project back 3072 -> 768

rng = np.random.default_rng(0)
W1, b1 = rng.normal(0, 0.02, (768, 3072)), np.zeros(3072)
W2, b2 = rng.normal(0, 0.02, (3072, 768)), np.zeros(768)
x = rng.normal(0, 1, (5, 768))      # 5 tokens, one row each

out = mlp(x, W1, b1, W2, b2)
print(np.round(gelu(np.array([-2.0, 0.0, 2.5])), 3))
print(out.shape, W1.size + b1.size + W2.size + b2.size)
print(np.allclose(out[2], mlp(x[2], W1, b1, W2, b2)))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. One token at a time · rewatch ▶ 0:08-0:21

1.1 B. A describes attention. C and D are wrong: it is one MLP, applied to every position.

1.2 False. The MLP sees only one token's vector. Moving information between tokens is attention's job.

1.3 (a) 5 times. (b) 1 set. One application per token, but it is the same MLP (the same weights) every time.

1.4 They are identical. The MLP sees only the vector and uses the same weights everywhere, so the same input always gives the same output.

2. Expand, bend, project back · rewatch ▶ 0:21-0:43

2.1 expand to $4\times$ → apply the nonlinear function → project back down. Expand, bend, project.

2.2 (a) 3,072. (b) 768. $4 \times 768 = 3,072$, and the projection brings it back to 768.

2.3 4,096 per token; 40,960 for 10 tokens. $4 \times 1,024 = 4,096$, and every token gets its own hidden vector: $10 \times 4,096$.

2.4 False. The 3,072 numbers exist only inside the MLP. The projection brings the vector back to 768.

3. Neurons as detectors · rewatch ▶ 0:44-1:00

3.1 2.5 and -3. The first (x). $1 \cdot 0.5 + 0 \cdot (-1) + 2 \cdot 1 = 2.5$ and $1 \cdot (-1) + 0 \cdot 2 + 2 \cdot (-1) = -3$. A large positive value means the vector points along w .

3.2 A. Aligned gives a large positive dot product; right angles gives 0; opposite gives a negative one.

3.3 True. w must be as long as the token vector (768). The 3,072 such vectors together are the expand matrix: $3,072 \times 768 = 2,359,296$ weights.

4. GELU: quiet unless the pattern shows up · rewatch ▶ 1:01-1:14

4.1 C. Large positive inputs pass through almost unchanged: $\text{GELU}(3) \approx 2.996$.

4.2 ≈ 2.5 , ≈ -0.05 , ≈ 4 , ≈ 0 . The first and the third fire. Exact values: 2.485, -0.045, 4.000, -0.004. Positive inputs pass, negative ones are squashed.

4.3 False. It squashes them *close to* zero (-0.05 at -2), not exactly to zero. Exactly 0 for every negative input is what ReLU does.

5. Why the bend matters · rewatch ► 1:14-1:26

5.1 (a) $W = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$. (b) Both give $\begin{bmatrix} 6 & 4 \end{bmatrix}$. $W_1x = \begin{bmatrix} 3 & 1 \end{bmatrix}$, then $W_2\begin{bmatrix} 3 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 4 \end{bmatrix}$; and $Wx = \begin{bmatrix} 2 + 4 & 1 + 3 \end{bmatrix} = \begin{bmatrix} 6 & 4 \end{bmatrix}$. Two matrices in a row act like one.

5.2 768×768 : 589,824 numbers. Without the bend, the 4.7 million weights could only do what 589,824 can: the expansion would be wasted.

5.3 False. The ten matrices collapse into one, which can still only fit straight lines.

5.4 B. A is the expand matrix, C comes at the very end of the model (episode 10), D is attention.

6. Recalling facts · rewatch ► 1:26-1:44

6.1 $\begin{bmatrix} 1 & 1 \end{bmatrix}$. $2 \cdot \begin{bmatrix} 1 & 0 \end{bmatrix} + 0 \cdot \begin{bmatrix} 0 & 1 \end{bmatrix} + 0.5 \cdot \begin{bmatrix} -2 & 2 \end{bmatrix} = \begin{bmatrix} 2 - 1 & 0 + 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix}$. The quiet neuron adds nothing.

6.2 B. The down projection adds a push: the vector keeps “Tower” and gains “Paris”.

6.3 False. In the MLP layers. Attention's job is moving information between tokens.

7. Where the weights are, in code · rewatch ► 1:44-2:14

7.1 $2/3$ (about 67 %). $4,718,592 / (4,718,592 + 2,359,296) = 2/3$.

7.2 MLP 8,388,608; attention 4,194,304. Yes, $2 \times$. $2 \times 1,024 \times 4,096$ and $4 \times 1,024 \times 1,024$. With the $4 \times$ rule the MLP always has $8 \times d \times d$ weights and attention $4 \times d \times d$.

7.3 B. $x @ W_1$ expands to 3,072 neuron values, and gelu bends each one. The output is only 768 after $@ W_2$.

7.4 (a) $\begin{bmatrix} -0.045 & 0.2485 \end{bmatrix}$ · (b) (5, 768) 4722432; h is (5, 3072) · (c) each token is processed on its own (a) -2 is squashed to nearly 0, 0 stays 0, and 2.5 passes almost unchanged. (b) Each of the 5 tokens gets 3,072 neuron values in h and is projected back to 768. The count is the 4,718,592 weights plus $3,072 + 768 = 3,840$ biases. (c) Running the MLP on token 2 alone gives exactly its row of the batch result: no token affects another, and all use the same weights (concept 1).